Web Technologies Lab — Solutions Q6–Q14

Complete exam-oriented HTML/CSS/JavaScript programs. Save each code block using the filename indicated.

Q6. CSS Animation — Lunar Eclipse

Save as lunar.html. This is a simplified visual effect.

```
<!DOCTYPE html>
<html><head><title>Lunar Eclipse</title>
<style>
body { background:#101525; color:white; text-align:center; }
.space { position:relative; width:300px; height:180px; margin:30px auto; }
.moon { position:absolute; width:80px; height:80px; background:#eee;
 border-radius:50%; top:50px; left:30px; }
.shadow { position:absolute; width:80px; height:80px; background:#101525;
 border-radius:50%; top:50px; left:30px; animation:eclipse 4s infinite alternate; }
@keyframes eclipse {
 from { transform:translateX(-100px); }
 to { transform:translateX(100px); }
}
</style></head><body><h2>Lunar Eclipse</h2>
<div class="space"><div class="moon"></div><div class="shadow"></div></div>
</body></html>
```

Key points: The Moon and shadow are circles; @keyframes moves the shadow. This is not a physically accurate eclipse simulation.

Q7. CSS Animation — Newton’s Cradle

Save as cradle.html. The two outer balls swing using CSS keyframes.

```
<!DOCTYPE html>
<html><head><title>Newton's Cradle</title>
<style>
body { text-align:center; background:#f2f2f2; }
.cradle { display:flex; justify-content:center; margin:60px auto; width:300px; border-top:8px solid #333; }
.ball { width:35px; height:35px; margin:0 2px; background:silver;
 border-radius:50%; position:relative; transform-origin:top center; }
.ball::before { content:""; position:absolute; width:2px; height:100px;
 background:#333; bottom:30px; left:50%; }
.ball:first-child { animation:leftSwing 1.5s infinite ease-in-out; }
.ball:last-child { animation:rightSwing 1.5s infinite ease-in-out; }
@keyframes leftSwing {
 0%,50%,100% { transform:rotate(0); } 25% { transform:rotate(35deg); }
}
@keyframes rightSwing {
 0%,50%,100% { transform:rotate(0); } 75% { transform:rotate(-35deg); }
}
</style></head><body><h2>Newton's Cradle</h2>
<div class="cradle"><div class="ball"></div><div class="ball"></div>
<div class="ball"></div><div class="ball"></div><div class="ball"></div></div>
</body></html>
```

Key points: Flexbox arranges the balls; ::before draws strings; transform-origin sets the pivot; keyframes rotate the outer balls. This is a visual approximation.

Q8. JavaScript Dialog Boxes

Save as q8.html. Demonstrates alert(), confirm(), and prompt().

```
<!DOCTYPE html>
<html><head><title>JavaScript Boxes</title>
<script>
function showBoxes() {
    alert("Welcome to JavaScript!");
    let choice = confirm("Do you want to continue?");
    if (choice) {
        let name = prompt("Enter your name:");
        if (name != null && name.trim() != "")
            alert("Welcome, " + name + "!");
        else
            alert("No name entered.");
    } else {
        alert("You cancelled the operation.");
    }
}
</script></head><body>
<h2>JavaScript Dialog Boxes</h2>
<button onclick="showBoxes()">Click Here</button>
</body></html>
```

Key points: alert displays a message; confirm returns true/false; prompt returns text or null. onclick triggers the function.

Q9. Open a New Browser Window with Custom Text

Save as q9.html.

```
<!DOCTYPE html>
<html><head><title>New Browser Window</title>
<script>
function openWindow() {
    let win = window.open("", "myWindow", "width=400,height=300");
    if (win) {
        win.document.write(`
          <html><head><title>Custom Page</title></head>
          <body><h2>Welcome!</h2>
          <p>This is a new browser window.</p></body></html>`);
        win.document.close();
    } else {
        alert("Please allow pop-ups.");
    }
}
</script></head><body>
<h2>Create Browser Window</h2>
<button onclick="openWindow()">Open Window</button>
</body></html>
```

Key points: window.open opens a window/tab; document.write writes HTML; document.close completes it. Pop-up blockers may prevent opening.

Q10. Dropdown Menu Linking Multiple Pages

Save as q10.html. Keep home.html, about.html, and contact.html in the same folder.

```
<!DOCTYPE html>
<html><head><title>Dropdown Navigation</title>
<script>
function navigatePage() {
    let page = document.getElementById("menu").value;
    if (page !== "") window.location.href = page;
}
</script></head><body>
<h2>Website Navigation</h2>
<label for="menu">Select a page:</label>
<select id="menu" onchange="navigatePage()">
  <option value="">-- Select Page --</option>
  <option value="home.html">Home</option>
  <option value="about.html">About</option>
  <option value="contact.html">Contact</option>
</select>
</body></html>
```

Key points: onchange triggers navigationPage; select.value gets the destination; window.location.href navigates.

Q11. Store Username and Last Access Date Using Cookies

Save as q11.html. Test on a local web server; cookies may behave inconsistently with file://.

```
<!DOCTYPE html>
<html><head><title>Welcome Using Cookies</title></head><body>
<h2 id="result"></h2>
<script>
function getCookie(name) {
    let cookies = document.cookie.split("; ");
    for (let cookie of cookies) {
        let [key, ...value] = cookie.split("=");
        if (key === name) return decodeURIComponent(value.join("="));
    }
    return "";
}
function setCookie(name, value) {
    document.cookie = name + "=" + encodeURIComponent(value) +
        "; max-age=2592000; path=/; SameSite=Lax";
}
function welcome() {
    let name = getCookie("username");
    let lastVisit = getCookie("lastVisit");
    if (!name) {
        name = prompt("Enter your name:");
        if (!name || !name.trim()) {
            document.getElementById("result").textContent = "Welcome, Guest!";
            return;
        }
        name = name.trim();
        setCookie("username", name);
    }
    let message = "Welcome, " + name + "!";
    if (lastVisit) message += " Last visited: " + lastVisit;
    else message += " This is your first visit.";
    document.getElementById("result").textContent = message;
    setCookie("lastVisit", new Date().toLocaleString());
}
window.onload = welcome;
</script></body></html>
```

Key points: Cookies store the username and last-visit time; window.onload runs the welcome function when the page loads.

Q12. Store and Display User Information Using Cookies

Save as q12.html. Enter name and email, click Save, then Display.

```
<!DOCTYPE html>
<html><head><title>User Information Cookies</title></head><body>
<h2>User Information</h2>
<input type="text" id="name" placeholder="Name">
<input type="email" id="email" placeholder="Email">
<button onclick="saveData()">Save</button>
<button onclick="showData()">Display</button>
<p id="result"></p>
<script>
function getCookie(name) {
    let cookies = document.cookie.split("; ");
    for (let cookie of cookies) {
        let [key, ...value] = cookie.split("=");
        if (key === name) return decodeURIComponent(value.join("="));
    }
    return "";
}
function saveData() {
    let name = document.getElementById("name").value.trim();
    let email = document.getElementById("email").value.trim();
    if (!name || !email) { alert("Enter both name and email"); return; }
    document.cookie = "name=" + encodeURIComponent(name) + "; path=/; SameSite=Lax";
    document.cookie = "email=" + encodeURIComponent(email) + "; path=/; SameSite=Lax";
    alert("Data saved successfully");
}
function showData() {
    let name = getCookie("name");
    let email = getCookie("email");
    document.getElementById("result").textContent =
        name && email ? "Name: " + name + " | Email: " + email
                      : "No user information found";
}
</script></body></html>
```

Key points: saveData validates and stores values; showData retrieves and displays them. These are session cookies because no expiry is specified.

Q13. Digital Clock Using setTimeout()

Save as q13.html.

```
<!DOCTYPE html>
<html><head><title>Digital Clock</title>
<style>body { text-align:center; font-family:Arial; }
#clock { font-size:40px; color:blue; }</style></head><body>
<h2>Digital Clock</h2><div id="clock"></div>
<script>
function showClock() {
    let now = new Date();
    let hours = String(now.getHours()).padStart(2, "0");
    let minutes = String(now.getMinutes()).padStart(2, "0");
    let seconds = String(now.getSeconds()).padStart(2, "0");
    document.getElementById("clock").textContent =
        hours + ":" + minutes + ":" + seconds;
    setTimeout(showClock, 1000);
}
showClock();
</script></body></html>
```

Key points: Date extracts current time; padStart formats two digits; setTimeout schedules another update after approximately one second.

Q14. Freehand Drawing with Canvas and Mouse Coordinates

Save as q14.html. Pointer events support mouse, touch, and pen.

```
<!DOCTYPE html>
<html><head><title>Canvas Drawing</title>
<style>
canvas { border:2px solid black; background:white; max-width:100%; touch-action:none; }
</style></head><body>
<h2>Freehand Drawing</h2>
<canvas id="board" width="500" height="300"></canvas>
<p id="coords">X: 0, Y: 0</p>
<button onclick="clearCanvas()">Clear</button>
<script>
const canvas = document.getElementById("board");
const ctx = canvas.getContext("2d");
let drawing = false;
ctx.lineWidth = 3;
ctx.lineCap = "round";
ctx.strokeStyle = "blue";
```

```
function getPosition(e) {
    const rect = canvas.getBoundingClientRect();
    return {
        x: (e.clientX - rect.left) * canvas.width / rect.width,
        y: (e.clientY - rect.top) * canvas.height / rect.height
    };
}
canvas.addEventListener("pointerdown", function(e) {
    drawing = true;
    canvas.setPointerCapture(e.pointerId);
    let p = getPosition(e);
    ctx.beginPath();
    ctx.moveTo(p.x, p.y);
});
canvas.addEventListener("pointermove", function(e) {
    let p = getPosition(e);
    document.getElementById("coords").textContent =
        "X: " + Math.round(p.x) + ", Y: " + Math.round(p.y);
    if (drawing) {
        ctx.lineTo(p.x, p.y);
        ctx.stroke();
    }
});
function stopDrawing() { drawing = false; }
canvas.addEventListener("pointerup", stopDrawing);
canvas.addEventListener("pointercancel", stopDrawing);
canvas.addEventListener("lostpointercapture", stopDrawing);
function clearCanvas() {
    ctx.clearRect(0, 0, canvas.width, canvas.height);
}
</script></body></html>
```

Key points: getContext('2d') enables drawing; pointer events start/update/stop; lineTo and stroke draw; clearRect clears the canvas.

Quick Revision Table

Question	Main concepts
Q6	CSS @keyframes, animation, transform
Q7	Flexbox, pseudo-element, rotation, keyframes
Q8	alert(), confirm(), prompt(), onclick
Q9	window.open(), document.write(), document.close()
Q10	select/option, onchange, window.location.href
Q11	Cookies, window.onload, Date
Q12	Cookie storage/retrieval, DOM inputs
Q13	Date object, setTimeout()
Q14	Canvas 2D API, pointer events, coordinates